

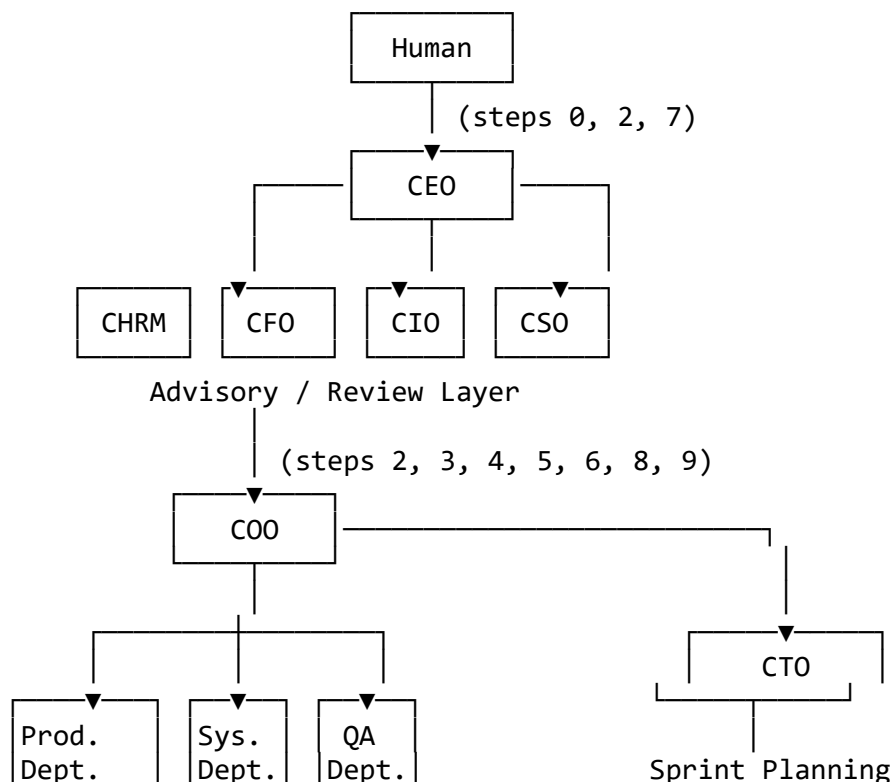
AIAT Organizational Architecture — Corporate Hierarchy MAS

TL;DR: A corporate-hierarchy multi-agent system (~20–40 agents across **11 teams**) where **C-Suite agents** (CEO, COO, CFO, CIO, CHRM, CSO, CTO) act as decision-makers and coordinators, **Department teams** (Production, System, QA, **DevOps**) execute the actual work, and a **Human-in-the-Loop** provides approval at critical gates. A **deterministic workflow controller** in the orchestrator-api owns all state transitions — agents emit events, the controller validates and advances state. Projects flow through a 14-step document lifecycle (PDR → CDR → RR → Infra Provisioning → Sprints → Retrospective → KPI) with parallel review fan-outs, CSO veto power, review circuit breakers, approval loops, and KPI-driven learning. Messages use a **single canonical MessageEnvelope** schema with idempotency keys, TTL, BlobRef for large payloads, and at-least-once delivery with effectively-once processing via Redis Streams consumer groups. All infra is \$0 self-hosted OSS; only LLM API costs apply.

Companion document: See `plan-masArchitectureUpgrade.prompt` for the infrastructure plan (Router, Redis Streams, Tool Service, Postgres/PgBouncer, MinIO, Docker Compose, Observability).

1. Organizational Topology

1.1 Agent Hierarchy



PM Workers	PM Eng.	Lead Test.
---------------	------------	---------------

Issue Tracking
KPI Analytics

1.2 Team Registry

Each agent or group runs as a **team-runner** container. C-Suite “offices” are single-agent (or small) teams. Departments are multi-worker teams.

Team ID	Admin Agent	Worker Agents	Purpose
exec_ceo	CEO	—	Human interface, top-level orchestration, project lifecycle
exec_coo	COO	—	Operations coordination, document lifecycle management
office_cfo	CFO	financial_analyst (0-2)	Financial feasibility, budget review, cost estimation
office_cio	CIO	tech_analyst (0-2)	Technical feasibility, technology stack review
office_chrm	CHRM	hr_analyst (0-1)	Resource availability, capacity planning, agent allocation
office_cso	CSO	security_analyst (0-2)	Security review, compliance, threat analysis
office_cto	CTO	sprint_planner (1), kpi_analyst (1)	Sprint planning, issue tracking, velocity, KPI learning
dept_production	Production PM	requirements_writer, planner, cost_estimator	PDR creation
dept_system	System PM	system_architect, solution_designer,	CDR creation (architecture,

		tech_writer	detailed design)
dept_qa	QA Lead	tester (1-3)	Quality assurance, test plan creation, validation
dept_devops	DevOps PM	devops_eng (1-2), sre_agent (0-1)	Infrastructure provisioning, CI/CD, monitoring, secrets management

Scaling note: Worker counts in parentheses are defaults. Each team YAML allows 0..N workers. For small projects a C-Suite office may have zero workers (the admin does analysis alone). For large projects, add workers dynamically (future: auto-scaling via CTO capacity analysis). **Total agents at defaults: ~25; max with all optional workers: ~40.**

1.3 Agent Role Mapping to MAS Roles

The existing MAS plan defines four roles: orchestrator, admin, worker, sub_agent. The corporate hierarchy maps as follows:

Corporate Role	MAS Role	Cross-Team Access	Notes
CEO	orchestrator	Anywhere	Only agent that interfaces with Human
COO	executive (new)	All departments + all C-Suite	Operational coordinator, document lifecycle owner
CFO, CIO, CHRM, CSO, CTO	c_suite (new)	Own team + CEO + COO + peer C-Suite (for reviews)	Advisory/review roles
Department PM	admin	Own team only	Manages workers within department
Worker	worker	Own team only	Executes

			tasks assigned by PM
Sub-Agent	sub_agent	Parent agent only	Spawned for subtasks

This requires extending the CommunicationPolicy (Phase 2 of the infrastructure plan) with two new roles and richer routing rules — see section 4 below.

2. Project Workflow — State Machine

2.1 Workflow Steps (14-step v2)

Key change: The **orchestrator-api** hosts a deterministic workflow controller that owns all state transitions. Agents do NOT directly mutate projects.state; they emit typed events (e.g., DOCUMENT_SUBMIT, REVIEW_RESPONSE, INFRA_READY). The controller validates each event against the transition table, persists the new state in Postgres, creates review sessions / approval gates as needed, and publishes work messages to agents via the Router.

Step 0: Human → orchestrator-api → CEO	Init project request
Step 1: CEO → CFO, CHRM, CSO, CIO	Parallel feasibility assessment
Step 2: CEO ← results → Human	Report feasibility; Human
approves → CEO → COO	
Step 3: COO → Production Dept. (financials, planning)	Create PDR (requirements,
Step 4: COO → CIO, CHRM, CSO, CFO	Parallel PDR review (via
review_sessions)	
Step 5: COO → System Dept. CDR	PDR + review comments → create
Step 6: COO → CEO → Human	Present final CDR
Step 7: Human decision	Approve / Cancel / Suggest
edits	
Step 8: Branch:	
- Approve → continue to step 9	
- Edits → CEO forwards to COO → redo steps 5→7	
- Cancel → CEO archives project, STOP	
Step 9: COO → RR document → CTO	Transform CDR into final
Requirements Review;	
sprints, issues, sub-issues	CTO creates milestones,
Step 10: CTO → DevOps Dept. environments, secrets, monitoring	INFRA_PROVISIONING: CI/CD,
	DevOps signals INFRA_READY
when done	
Step 11: CTO → Sprint execution	Assign workers, track
velocity, report remaining	

first dev sprint)

Step 12: CTO → Sprint retrospective
KPIs, identify issues,

Step 13: CTO → KPI persistence → CEO
agent_profiles,

(BLOCKED until INFRA_READY for

After each sprint: compute

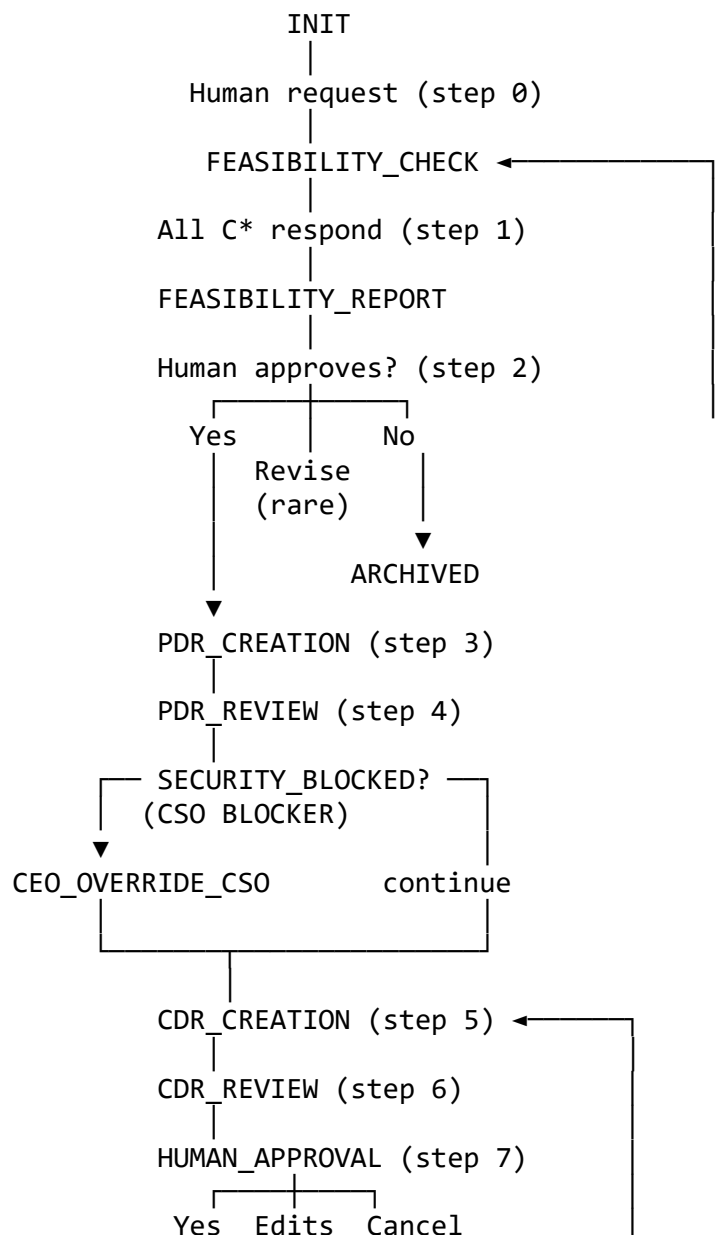
adjust next sprint plan

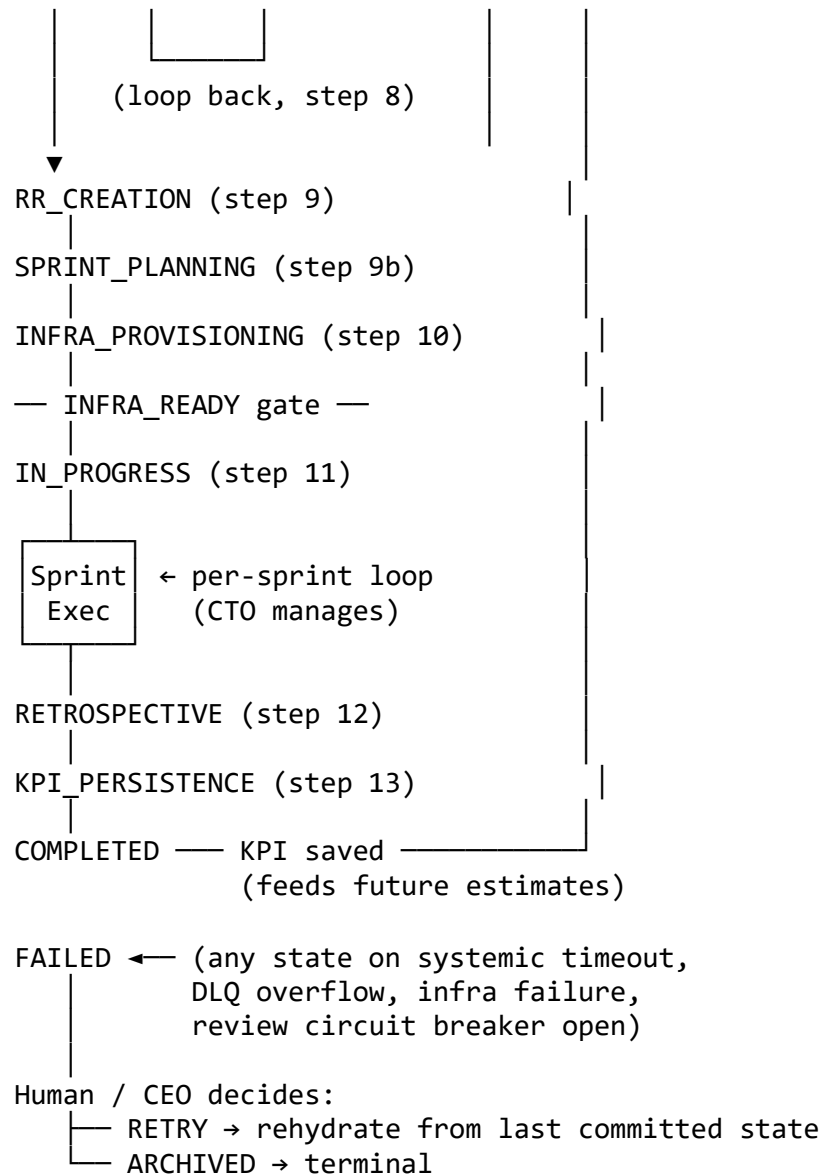
Save all KPI data, update

final report to CEO → Human

Watchdog: The controller runs a background job — if no workflow event arrives for a configurable timeout (default 1 hour), it publishes a DIRECTIVE to the current state's responsible agent AND notifies the CEO via ESCALATION.

2.2 Project State Machine





2.3 State Definitions

Controller rule: Only the orchestrator-api controller may write to `projects.state`. Agents emit events; the controller validates transitions against the table below and persists the new state atomically in Postgres (with `updated_at` timestamp). Invalid transitions are rejected and logged.

State	Responsible Agent	Controller Actions	Exit Event
INIT	CEO	Creates project record in Postgres	project_created
FEASIBILITY_CHECK	CEO	Controller creates review_session for feasibility; fans out REVIEW_REQUEST to CFO,	all_reviews_in

		CIO, CHRM, CSO	
FEASIBILITY_REPORT	CEO	Controller opens FEASIBILITY approval gate	human_approved / human_rejected
PDR_CREATION	COO → Production PM	Controller publishes TASK to Production dept	document_submitted (DOCUMENT_SUBMIT event)
PDR_REVIEW	COO	Controller creates review_session; fans out REVIEW_REQUEST for PDR. CSO veto: any CSO BLOCKER → SECURITY_BLOCKED substate. Circuit breaker: ≥2 reviewer timeouts → FAILED(REVIEW_CIRCUIT_OPEN)	all_reviews_in
SECURITY_BLOCKED	CEO	Workflow halted. CEO must emit CEO_OVERRIDE_CSO (with justification, logged to Postgres audit) or human_cancelled	ceo_override_cso / human_cancelled
CDR_CREATION	COO → System PM	Controller publishes TASK to System dept with PDR + review comments	document_submitted
CDR_REVIEW	COO → CEO	Controller aggregates CDR, publishes to CEO for Human presentation	cdr_presented
HUMAN_APPROVAL	CEO	Controller opens CDR_APPROVAL gate	human_approved / human_edits / human_cancelled
RR_CREATION	COO	Controller publishes TASK to COO to transform CDR → RR	rr_submitted
SPRINT_PLANNING	CTO	Controller publishes TASK to CTO to decompose RR into sprints/issues	sprints_created
INFRA_PROVISIONING	CTO → DevOps PM	Controller publishes TASK to DevOps dept; blocks sprint execution until INFRA_READY	infra_ready
IN_PROGRESS	CTO	Controller monitors sprint lifecycle; per-sprint loop via CTO	all_sprints_done

RETROSP ECTIVE	CTO	Controller publishes TASK to CTO to compute sprint KPIs, identify issues	retrospective_ complete
KPI_PER SISTENC E	CTO → CEO	Controller triggers KPI computation, agent_profiles update, final report to CEO → Human	kpi_saved
COMPLET ED	CEO	Project complete; all data preserved	archived
ARCHIVE D	—	Terminal state. Project cancelled or completed; all data preserved for KPI	—
FAILED	CEO	Entered on systemic timeout, DLQ overflow, infra failure, or review circuit breaker. Human/CEO decides RETRY (rehydrate from last committed state) or ARCHIVE	retry / human_cancelle d

FAILED state details: The controller records `failure_reason` (enum: REVIEW_CIRCUIT_OPEN, WATCHDOG_TIMEOUT, DLQ_OVERFLOW, INFRA_FAILURE, AGENT_BUDGET_EXHAUSTED) and `failed_from_state` (the state that was active when failure occurred). On RETRY, the controller transitions back to `failed_from_state` and re-publishes the work message.

3. Unified Message Protocol

3.1 Canonical MessageEnvelope

Key change: Replace the previous dual-schema (RouterEnvelope wrapping Message) with a **single canonical MessageEnvelope**. All agents, the controller, and the router use this one schema. The router validates it; Redis Streams store it; agents parse it.

```
class MessageEnvelope(BaseModel):
    """Single canonical message schema for the entire MAS."""
    # Identity & idempotency
    message_id: UUID                                # Primary idempotency key (UUID,
    set by sender)
    correlation_id: UUID                            # Groups related messages in a
    workflow step
    parent_id: UUID | None = None                  # Links to the message that spawned
    this one

    # Routing
```



```

msg_type: MessageType           # See expanded enum below
sender_id: str                  # agent_id of sender
sender_role: AgentRole          # One of 6 roles (see section 4)
sender_team: str                # team_id
recipient_id: str | None = None # Target agent_id (None if
broadcasting to team)
recipient_team: str | None = None # Target team_id (required if
recipient_id is None)

# Project context
project_id: UUID | None = None # Mandatory after INIT state
(validators enforces)

# Timing & delivery
timestamp: datetime            # UTC, set by sender
ttl_seconds: int = 3600        # Default 1 hour; router drops
expired messages
retry_count: int = 0           # Incremented by router on
redelivery via XAUTOCLAIM
ack_required: bool = True      # If False, fire-and-forget (use
sparingly: HEARTBEAT, BROADCAST)

# Content
payload: dict[str, Any]        # Small structured data (must pass
size validator)
blob_ref: BlobRef | None = None # Pointer to MinIO for large
payloads
budget: TaskBudget | None = None # Embedded for TASK-like msg_types

# Validators
@validator("payload")
def enforce_max_size(cls, v):
    import json
    if len(json.dumps(v).encode()) > MAX_MESSAGE_BYTES:
        raise ValueError(f"payload exceeds {MAX_MESSAGE_BYTES} bytes; use
blob_ref")
    return v

@validator("project_id")
def require_project_id_after_init(cls, v, values):
    """project_id is mandatory for all msg_types except SHUTDOWN and
HEARTBEAT. """
    exempt = {MessageType.SHUTDOWN, MessageType.HEARTBEAT}
    if values.get("msg_type") not in exempt and v is None:
        raise ValueError("project_id required for this msg_type")
    return v

MAX_MESSAGE_BYTES = 64 * 1024 # 64 KB

```

```

class BlobRef(BaseModel):
    """Pointer to large content stored in MinIO/S3."""
    minio_key: str # e.g., "mas-
agents/{project_id}/documents/cdr_v2.json"
    sha256: str # Content hash for integrity
    verification
    size_bytes: int # For budget tracking and display

```

```

class AgentRole(str, Enum):
    ORCHESTRATOR = "orchestrator" # CEO
    EXECUTIVE = "executive" # COO
    C_SUITE = "c_suite" # CFO, CIO, CHRM, CSO, CTO
    ADMIN = "admin" # Department PM
    WORKER = "worker" # Department worker
    SUB_AGENT = "sub_agent" # Spawned subtask agent

```

3.2 Expanded MessageType Enum

```

class MessageType(str, Enum):
    # Core work
    TASK = "TASK"
    RESULT = "RESULT"
    QUERY = "QUERY"
    RESPONSE = "RESPONSE"
    BROADCAST = "BROADCAST"
    ADMIN_TASK = "ADMIN_TASK"
    ADMIN_REPLY = "ADMIN_REPLY"
    SHUTDOWN = "SHUTDOWN"

    # Document lifecycle
    DOCUMENT_SUBMIT = "DOCUMENT_SUBMIT"
    DOCUMENT_REVISION = "DOCUMENT_REVISION"

    # Review workflow
    REVIEW_REQUEST = "REVIEW_REQUEST"
    REVIEW_RESPONSE = "REVIEW_RESPONSE"

    # Human-in-the-loop
    APPROVAL_REQUEST = "APPROVAL_REQUEST"
    APPROVAL_RESPONSE = "APPROVAL_RESPONSE"

    # Sprint management
    SPRINT_PLAN = "SPRINT_PLAN"
    SPRINT_REPORT = "SPRINT_REPORT"
    ISSUE_ASSIGN = "ISSUE_ASSIGN"
    ISSUE_COMPLETE = "ISSUE_COMPLETE"

    # DevOps / Infrastructure

```

```

INFRA_READY = "INFRA_READY"           # DevOps → controller: infra
provisioned

# Hierarchy & control
ESCALATION = "ESCALATION"
DIRECTIVE = "DIRECTIVE"

# System
HEARTBEAT = "HEARTBEAT"               # Router ↔ agent liveness
ACK = "ACK"                           # Explicit message acknowledgment
(ws protocol)

```

3.3 WS Subscribe Protocol

The router delivers messages to agents over WebSocket using this frame format:

```

{
  "entry_id": "1708900000000-0",
  "envelope": { /* full MessageEnvelope JSON */ },
  "attempt": 1
}

```

Agent responses: - {"type": "ACK", "entry_id": "1708900000000-0"} → router calls XACK; marks delivered - {"type": "NACK", "entry_id": "1708900000000-0", "reason": "..."} → stays in PEL for redelivery - Router sends PING frames every 15s; agent must reply PONG within 10s or connection is considered dead

3.4 Domain Models (Pydantic)

```

class ProjectDocument(BaseModel):
    """A versioned project document (PDR, CDR, or RR)."""
    document_id: UUID
    project_id: UUID
    doc_type: Literal["PDR", "CDR", "RR"]
    version: int                    # Incremented on each revision
    title: str
    content_ref: str                # MinIO key (large content stored
as blob)
    sections: dict[str, Any]        # Structured metadata (requirements
list, etc.)
    created_by: str                 # agent_id
    created_at: datetime
    status: Literal["DRAFT", "IN_REVIEW", "APPROVED", "REJECTED",
"SUPERSEDED"]
    parent_document_id: UUID | None # PDR that this CDR was based on,
etc.

class ReviewComment(BaseModel):
    """A single review comment from a C-Suite reviewer."""
    reviewer_id: str                # agent_id (e.g., "cfo", "cso")
    reviewer_role: str              # e.g., "CFO", "CSO"

```

```

severity: Literal["BLOCKER", "MAJOR", "MINOR", "SUGGESTION", "APPROVED"]
section: str | None # Which section the comment targets
comment: str
suggested_fix: str | None
created_at: datetime

class ReviewSummary(BaseModel):
    """Aggregated review result for a document."""
    document_id: UUID
    document_version: int
    reviews: list[ReviewComment]
    overall_verdict: Literal["APPROVED", "NEEDS_REVISION", "REJECTED"]
    blocker_count: int
    major_count: int
    minor_count: int

class FeasibilityReport(BaseModel):
    """Aggregated feasibility assessment from step 1."""
    project_id: UUID
    financial: ReviewComment # From CFO
    technical: ReviewComment # From CIO
    resources: ReviewComment # From CHRM
    security: ReviewComment # From CSO
    overall_feasible: bool
    estimated_cost: float | None
    estimated_duration_days: int | None
    risk_level: Literal["LOW", "MEDIUM", "HIGH", "CRITICAL"]

class HumanDecision(BaseModel):
    """Human's decision at an approval gate."""
    project_id: UUID
    gate_type: Literal["FEASIBILITY", "CDR_APPROVAL"]
    decision: Literal["APPROVE", "REJECT", "EDIT"]
    comments: str | None
    edits: list[str] | None # Specific edit requests
    decided_at: datetime

class Sprint(BaseModel):
    """A sprint created by CTO from the RR."""
    sprint_id: UUID
    project_id: UUID
    name: str
    milestone: str
    start_date: datetime
    end_date: datetime
    status: Literal["PLANNED", "ACTIVE", "COMPLETED", "CANCELLED"]
    issues: list["Issue"]

class Issue(BaseModel):

```

```

"""A work item within a sprint."""
issue_id: UUID
sprint_id: UUID
project_id: UUID
title: str
description: str
issue_type: Literal["FEATURE", "TEST", "QA", "BUGFIX", "DOCS", "INFRA"]
assignee_team: str # team_id
assignee_agent: str | None # agent_id (assigned by PM)
priority: Literal["CRITICAL", "HIGH", "MEDIUM", "LOW"]
estimate_hours: float | None
actual_hours: float | None
status: Literal["TODO", "IN_PROGRESS", "IN_REVIEW", "DONE", "BLOCKED"]
parent_issue_id: UUID | None # For sub-issues
dependencies: list[UUID] # Issue IDs this depends on

class KPISnapshot(BaseModel):
    """Performance metrics captured per sprint/project."""
    project_id: UUID
    sprint_id: UUID | None
    agent_id: str
    metric_type: Literal[
        "ESTIMATION_ACCURACY", # estimated vs actual hours
        "TASK_COMPLETION_RATE", # done / total
        "REVIEW_PASS_RATE", # approved on first review / total
        "VELOCITY", # story points per sprint
        "DEFECT_RATE", # bugs found in QA / features delivered
        "REWORK_RATE", # CDR revision loops
        "BUDGET_ADHERENCE", # actual cost vs estimated cost
        "RESOURCE_UTILIZATION", # hours_worked / hours_available
        "INFRA_LEAD_TIME", # time from INFRA_PROVISIONING →
INFRA_READY
    ]
    value: float
    context: dict[str, Any] | None # Additional context for Learning
    measured_at: datetime

class AgentProfile(BaseModel):
    """Per-agent rolling performance metrics for estimation correction."""
    agent_id: str
    agent_role: str
    avg_estimation_accuracy: float # Rolling average
    estimation_bias: float # Positive = overestimates,
negative = underestimates
    correction_factor: float # Multiply raw estimates by this
(default 1.0)
    avg_velocity: float | None # Story points per sprint (workers
only)
    budget_adherence_avg: float | None # Historical cost ratio

```

```

confidence: float           # 0.0-1.0; increases with more data
points
sample_count: int          # Number of data points backing
this profile
last_updated: datetime

```

4. Enhanced Communication Policy

4.1 Role-Based Communication Matrix

The policy engine must enforce the corporate hierarchy. Replace the flat 4-role model with:

	Can Send To:					
	Human	CEO	C-Suite	Dept PM	Worker	Sub-Agent
CEO	✓	—	✓	via COO	X	X
COO	X	✓	✓	✓	X	X
C-Suite (CFO,CIO, CHRM,CSO, CTO)	X	✓	✓* (review)	X	own team	X
Dept PM	X	X	COO only	X	own team	X
Worker	X	X	X	own PM	own team	own
Sub-Agent	X	X	X	X	parent	X

✓* = C-Suite can communicate with each other ONLY during review workflows
(message types: REVIEW_REQUEST, REVIEW_RESPONSE)

4.2 Policy Rules (Updated)

```

POLICY_RULES = {
  "orchestrator": {
    # CEO role
    "allowed_targets": ["*"],           # Can reach anyone
    "allowed_msg_types": ["*"],        # Can send any message
    type
    "human_interface": True,           # Only role that talks
    to Human

```

```

    "allowed_tools": ["*"],                                # Full tool access
  },
  "executive": {
    # COO role
    "allowed_targets": [
      "role:orchestrator",                                # → CEO
      "role:c_suite",                                     # → CFO, CIO, CHRM, CSO,
CTO
      "role:admin",                                       # → Department PMs
      (direct)
    ],
    "allowed_msg_types": [
      "TASK", "RESULT", "ADMIN_TASK", "ADMIN_REPLY",
      "REVIEW_REQUEST", "REVIEW_RESPONSE",
      "DOCUMENT_SUBMIT", "DOCUMENT_REVISION",
      "DIRECTIVE", "BROADCAST", "SHUTDOWN",
    ],
    "allowed_tools": [
      "document.*", "review.*", "project.status",
      "blob.*", "kpi.query_history",
    ],
  },
  "c_suite": {
    # CFO, CIO, CHRM, CSO, CTO
    "allowed_targets": [
      "role:orchestrator",                                # → CEO (report up)
      "role:executive",                                   # → COO (report up)
      "role:c_suite",                                     # → peer C* (review
workflows only)
      "team:own",                                         # → own workers
    ],
    "allowed_msg_types": [
      "TASK", "RESULT", "QUERY", "RESPONSE",
      "REVIEW_REQUEST", "REVIEW_RESPONSE",
      "SPRINT_PLAN", "SPRINT_REPORT",
      "ISSUE_ASSIGN", "ISSUE_COMPLETE",
      "ADMIN_TASK", "ADMIN_REPLY",
      "ESCALATION", "INFRA_READY",
    ],
    "cross_team_msg_types": [
      "REVIEW_REQUEST", "REVIEW_RESPONSE",                # Only these cross-team
      "ESCALATION", "ADMIN_REPLY",
      "SPRINT_PLAN",                                     # CTO → dept PMs cross-
team
    ],
    "allowed_tools": [
      # Per-role overrides applied at team YAML level
      "document.get_latest", "document.list",
      "blob.download", "blob.list",
      "web_search", "web_fetch",

```

```

    ],
    "cto_extra_tools": [
        "sprint.*", "issue.*",
        "kpi.compute_sprint", "kpi.compute_project",
        "kpi.query_history", "kpi.update_agent_profile",
    ],
},
"admin": {
    # Department PM
    "allowed_targets": [
        "role:executive",           # → COO (report up)
        "role:c_suite:cto",         # → CTO (sprint reports)
        "team:own",                 # → own workers
    ],
    "allowed_msg_types": [
        "TASK", "RESULT", "QUERY", "RESPONSE",
        "DOCUMENT_SUBMIT", "DOCUMENT_REVISION",
        "ISSUE_ASSIGN", "ISSUE_COMPLETE",
        "SPRINT_REPORT", "ESCALATION", "INFRA_READY",
    ],
    "allowed_tools": [
        "document.create_draft", "document.submit", "document.revise",
        "document.get_latest", "document.list",
        "blob.*",
        "issue.update_status",
    ],
    "devops_pm_extra_tools": [
        "infra.provision", "cicd.configure",
        "monitoring.setup", "secrets.manage",
        "infra.ready_signal",
    ],
},
"worker": {
    "allowed_targets": ["team:own"],
    "allowed_msg_types": [
        "TASK", "RESULT", "QUERY", "RESPONSE",
        "ISSUE_COMPLETE", "ESCALATION",
    ],
    "allowed_tools": [
        "document.get_latest", "document.list",
        "blob.upload", "blob.download", "blob.list",
        "web_search", "web_fetch",
    ],
    "blocked_tools": [
        "project.*", "approval.*",
        "review.start_session", "review.aggregate",
        "sprint.create", "sprint.activate",
    ],
},
"sub_agent": {

```



```

    "allowed_targets": ["parent:only"],
    "allowed_msg_types": ["RESULT", "QUERY"],
    "allowed_tools": ["blob.download", "web_search"],
  },
}

```

Tool permissions enforcement: The tool-service validates (sender_role, tool_name) on every call. Workers cannot call project.transition, approval.*, review.start_session, etc. The controller (orchestrator-api) and CEO/COO/CTO have the broadest access.

4.3 Chain of Command Enforcement

Messages that skip hierarchy levels are **rejected** by the router: - A worker cannot message the CEO directly — must escalate to PM → COO → CEO - A department PM cannot message CFO directly — must go through COO - Only the CEO can trigger Human-facing API calls

Exception: ESCALATION messages can skip one level up (worker → PM, PM → COO, C-Suite → CEO) without going through the full chain.

4.4 Router / Redis Streams Hardening

This section specifies **how** the message-router uses Redis Streams. Refer to the infrastructure plan for low-level connection details; this section adds organisational-architecture-specific hardening.

4.4.1 Consumer Groups & Delivery

```

# Per-team stream: stream:{team_id}    (e.g., stream:exec_ceo)
# Consumer group per team:  group:{team_id}

```

```

XREADGROUP GROUP group:exec_ceo ceo_consumer
COUNT 10 BLOCK 5000 STREAMS stream:exec_ceo >

```

```

# After processing:
XACK stream:exec_ceo group:exec_ceo <message_id>

```

- Every team has **one** Redis Stream and **one** consumer group.
- The team-runner process is the sole consumer in that group.
- Messages pending > 120 s are auto-reclaimed:

```

XAUTOCLAIM stream:{team_id} group:{team_id} {consumer_id} 120000 0-0
# Returns reclaimed entries; router increments retry_count in the envelope

```

4.4.2 Dead-Letter Queue (DLQ)

A message enters the DLQ when **any** of: - retry_count >= max_attempts (default max_attempts = 3) - now() - timestamp > ttl_seconds (TTL expired before first or subsequent delivery)

DLQ flow:

XAUTOCLAIM returns entry with `delivery_count ≥ 3`

- Router writes row to Postgres `dead_letters` table (see §10)
- Router XACK + XDEL the original stream entry
- Router publishes `SYSTEM_EVENT { event: "DLQ_ENTRY", message_id }` to `stream:orchestrator` (CEO is notified)

Clarification: The infrastructure plan's `pending_messages` Postgres table is **renamed** to `dead_letters`. In-flight messages are tracked solely by the Redis PEL (Pending Entries List). Only messages that exhaust retries land in Postgres for forensic review.

4.4.3 Publish-Side Idempotency

Before XADD, router checks:

```
dedupe_key = f"dedupe:{envelope.message_id}"
if await redis.set(dedupe_key, 1, nx=True, ex=300):
    # First time – publish
    await redis.xadd(f"stream:{recipient_team}", envelope.dict())
else:
    # Duplicate – silently discard, return ACK to sender
    pass
```

- Deduplication window: **300 s** (5 minutes).
- Key format: `dedupe:{message_id}` — auto-expires.

4.4.4 Consume-Side Idempotency

Each consumer (team-runner) maintains a local LRU set (size 1 000) of recently processed `message_id` values. On delivery: 1. If `message_id` in LRU → XACK immediately, skip processing. 2. Else → process → add to LRU → XACK.

This shields against XAUTOCLAIM re-delivery races.

4.4.5 Stream Trimming

Cron job inside router (every 60 s):

```
for stream in all_team_streams:
    await redis.xtrim(stream, maxlen=50_000, approximate=True)
```

- MAXLEN ~ 50000 keeps memory bounded while preserving recent history for debugging.
- Old entries beyond 50 000 are already ACK'd; no data loss.

4.4.6 Redis ACL

router_user – used by message-router

```
ACL SETUSER router_user on >$ROUTER_PASS ~stream:* ~dedupe:* ~heartbeat:*
+xadd +xreadgroup +xack +xautoclaim +xdel +xtrim +xlen +xinfo
+set +get +del +expire +ping
```

toolcache_user – used by tool-service for caching

```
ACL SETUSER toolcache_user on >$TOOL_PASS ~tool_cache:*
```

+get +set +del +expire +ping

Disable default user
ACL SETUSER default off

- **Principle of least privilege:** each service gets only the commands and key patterns it needs.
 - Credentials are injected via Docker secrets / environment variables; never hard-coded.
-

5. C-Suite Agent Specifications

5.1 CEO — Chief Executive Officer

System prompt focus: Strategic decision-making, human communication, feasibility aggregation, project lifecycle management, controller-aware state orchestration.

Capabilities: - Receives project requests from Human (via orchestrator API) - Fans out feasibility checks to C-Suite (parallel REVIEW_REQUEST) - Aggregates feasibility reports into a summary for Human - Presents documents (CDR) to Human for approval - Emits state-transition events to the **deterministic workflow controller** in orchestrator-api (does NOT write projects.state directly) - Archives/cancels projects on Human request - Final authority on all decisions (except Human override) - Can issue CEO_OVERRIDE_CSO to unblock a SECURITY_BLOCKED project (justification logged in approval_gates)

Tools: project.create, project.status, project.transition (via controller), human.notify, human.await_decision, review.aggregate, approval.override_cso

Budget defaults: High LLM budget (complex reasoning), low tool budget (mostly coordination)

5.2 COO — Chief Operating Officer

System prompt focus: Operational coordination, document lifecycle management, department tasking, review orchestration.

Capabilities: - Receives directives from CEO with full context - Tasks Production department to create PDR - Orchestrates parallel PDR review across C-Suite - Tasks System department to create CDR (with PDR + review comments) - Sends CDR up to CEO for Human approval - Handles revision loops (step 5→7 redo) - Transforms final CDR into RR, sends to CTO - Monitors department progress

Tools: document_create, document_get, review_aggregate, department_task, project_status_update

Budget defaults: High LLM budget, medium tool budget

5.3 CFO — Chief Financial Officer

System prompt focus: Financial analysis, budget estimation, cost/benefit assessment, ROI calculation.

Reviews for: Financial feasibility (step 1), PDR budget/cost sections (step 4)

Capabilities: - Assesses project financial viability - Reviews cost estimates in PDR - Flags budget risks - Provides cost/return analysis - Workers (if any): financial analysts for detailed modeling

Tools: cost_estimate, budget_check, roi_calculate, market_research

5.4 CIO — Chief Information Officer

System prompt focus: Technical feasibility, technology stack assessment, integration analysis, information systems review.

Reviews for: Technical feasibility (step 1), PDR technical sections (step 4)

Capabilities: - Assesses technical viability and complexity - Reviews technology choices - Identifies integration risks - Evaluates whether existing infrastructure supports the project - Workers (if any): tech analysts for deep technical research

Tools: tech_stack_analyze, integration_check, web_search, code_analyze

5.5 CHRM — Chief Human Resource Manager

System prompt focus: Resource planning, agent capacity assessment, team composition, workload balancing.

Reviews for: Resource feasibility (step 1), PDR resource/planning sections (step 4)

Capabilities: - Assesses available agent capacity - Reviews project staffing requirements - Identifies resource conflicts across projects - Recommends team composition - Tracks agent utilization metrics

Tools: capacity_check, agent_registry_query, workload_report, team_recommend

5.6 CSO — Chief Security Officer

System prompt focus: Security review, threat analysis, compliance checks, data protection, **veto authority**.

Reviews for: Security feasibility (step 1), PDR security sections (step 4)

Capabilities: - Security threat assessment for proposed architecture - Reviews data handling and access control requirements - Compliance validation - Identifies security risks in design - **Veto power:** Any CSO review with severity BLOCKER triggers SECURITY_BLOCKED sub-state. The project cannot advance until the blocker is resolved or the CEO issues CEO_OVERRIDE_CSO (with mandatory justification) - Workers (if any): security analysts for detailed threat modeling

Tools: threat_model, compliance_check, security_scan, risk_assess, review.submit_veto

CSO Veto Protocol: 1. CSO submits REVIEW_RESPONSE with severity: BLOCKER and veto: true 2. Controller transitions project to SECURITY_BLOCKED sub-state 3. COO is notified; COO escalates to CEO 4. CEO either: a. Tasks the originating department to address the concern → re-review b. Issues CEO_OVERRIDE_CS0 with justification field (persisted in approval_gates) 5. Only after (a) CSO approves or (b) CEO overrides does the controller resume the prior state

5.7 CTO — Chief Technology Officer

System prompt focus: Sprint planning, issue decomposition, milestone creation, velocity tracking, KPI analysis, estimation improvement, **DevOps coordination**.

Capabilities: - Receives final RR from COO - Decomposes into milestones → sprints → issues → sub-issues - Each issue gets: type (feature/test/QA/docs/**infra**), estimated hours, priority, dependencies, assignee team - Assigns issue batches to department PMs — including **DevOps PM** for INFRA type issues (via SPRINT_PLAN) - Sends INFRA_PROVISIONING signal to DevOps PM before first dev sprint - Waits for INFRA_READY gate from DevOps PM before activating dev sprints - Before each sprint: fetches remaining issues from last sprint, restructures - After each sprint: collects SPRINT_REPORT from PMs (including DevOps PM) - Computes KPI metrics: estimation accuracy, velocity, defect rate, rework rate, **infra lead time** - Saves KPI data to Postgres for future estimation learning - Uses agent profiles (agent_profiles table) with correction_factor to adjust per-agent estimates - Uses historical KPI data to improve estimates on new projects

Tools: sprint.create, issue.create, issue.decompose, kpi.compute, kpi.query_history, kpi.update_agent_profile, velocity.report, estimation.adjust

Budget defaults: Medium LLM budget, high tool budget (lots of DB operations)

6. Department Specifications

6.1 Production Department (dept_production)

Purpose: Create the Primary Design Review (PDR) document.

Admin: Production PM **Workers:** requirements_writer, planner, cost_estimator

Input: Project brief + feasibility data from CEO/COO (step 3) **Output:** PDR document containing: - Functional & non-functional requirements - Technical requirements - Financial plan and budget breakdown - Project timeline and milestones (initial) - Resource plan - Risk register

Workflow internal to department: 1. PM receives TASK from COO, decomposes into subtasks 2. requirements_writer → drafts functional/non-functional requirements 3. planner → creates timeline, milestones, resource plan 4. cost_estimator → creates

financial plan and budget breakdown 5. PM aggregates into PDR document, stores in MinIO, submits to COO

6.2 System Department (dept_system)

Purpose: Create the Critical Design Review (CDR) document.

Admin: System PM **Workers:** system_architect, solution_designer, tech_writer

Input: PDR + all C-Suite review comments (step 5) **Output:** CDR document containing: - System architecture (components, interactions, data flow) - Detailed technical design per component - API specifications - Database schemas - Infrastructure requirements - Security architecture - Enhanced requirements (addressing all review comments) - Full diagrams (stored as blobs in MinIO)

Workflow internal to department: 1. PM receives PDR + review comments, decomposes into subtasks 2. system_architect → designs overall architecture, component diagram 3. solution_designer → detailed design per component, API specs, DB schemas 4. tech_writer → assembles full CDR with diagrams, ensures all review comments addressed 5. PM reviews, aggregates into CDR document, stores in MinIO, submits to COO

6.3 QA Department (dept_qa)

Purpose: Quality assurance during sprint execution (step 10).

Admin: QA Lead **Workers:** tester (1-3)

Activated during: Sprint execution (step 10), receives QA-type issues from CTO

Responsibilities: - Create test plans from requirements - Execute test cases - Report defects as new issues - Validate fixes - Acceptance testing per milestone

6.4 DevOps Department (dept_devops) — v1 Critical Path

Purpose: Infrastructure provisioning, CI/CD pipeline configuration, environment setup, monitoring, and secrets management. **Must signal INFRA_READY before dev sprints begin.**

Admin: DevOps PM **Workers:** devops_eng, sre_agent

Activated during: Step 8 (INFRA_PROVISIONING) and sprint execution (step 10 for INFRA-type issues)

Input: RR infrastructure requirements + CTO-issued INFRA-type issues (via SPRINT_PLAN)

Output: Provisioned environments, CI/CD pipelines, monitoring dashboards, INFRA_READY signal

Workflow internal to department: 1. DevOps PM receives SPRINT_PLAN from CTO with INFRA type issues 2. devops_eng → provisions environments (dev, staging, prod stubs), configures CI/CD pipelines 3. sre_agent → sets up monitoring, alerting thresholds, logging infrastructure 4. DevOps PM → manages secrets rotation, validates environment health 5.

DevOps PM → runs smoke tests against provisioned infra 6. DevOps PM → sends INFRA_READY message to CTO (triggers controller gate)

Tools: `infra.provision`, `cicd.configure`, `monitoring.setup`, `secrets.manage`, `infra.ready_signal`, `blob.upload`, `blob.download`

INFRA_READY Gate: - CTO **cannot** activate dev sprint issues until DevOps PM sends INFRA_READY - Controller enforces: INFRA_PROVISIONING → INFRA_READY event → IN_PROGRESS - If DevOps exceeds SLA (configurable, default 30 min), watchdog escalates to CTO → CEO

During sprints: DevOps receives ongoing INFRA issues (CI fixes, env scaling, deployment tasks) like any other department.

7. Document Lifecycle Management

7.1 Document States

DRAFT → IN_REVIEW → { APPROVED | NEEDS_REVISION }

↓

REVISION (new version) → IN_REVIEW → ...

APPROVED → SUPERSEDED (when a new version is approved)

7.2 Document Storage Strategy

- **Metadata** → Postgres documents table (searchable, versioned)
- **Content body** → MinIO blob (`mas-agents/{project_id}/documents/{doc_type}_v{version}.json`)
- **Diagrams / attachments** → MinIO blob (`mas-agents/{project_id}/artifacts/{filename}`)
- **Retrospectives** → MinIO blob (`mas-agents/{project_id}/retrospectives/sprint_{n}.json`)
- **Messages carry references only** — `blob_ref: {bucket, key, sha256}` via BlobRef model (per §3.1 payload size rules: ≤64 KB inline, else BlobRef)

7.3 Review Process (Parallel Fan-Out / Fan-In)

COO sends REVIEW_REQUEST to N reviewers (parallel)

├── CFO reviews (async) → REVIEW_RESPONSE
├── CIO reviews (async) → REVIEW_RESPONSE
├── CHRM reviews (async) → REVIEW_RESPONSE
└── CSO reviews (async) → REVIEW_RESPONSE

COO collects all N responses (fan-in with timeout)

↓

ReviewSummary aggregated

— CSO BLOCKER with veto?	→ SECURITY_BLOCKED (see §5.6)
— Any other BLOCKER?	→ overall = NEEDS_REVISION
— >2 MAJOR?	→ overall = NEEDS_REVISION
— Otherwise	→ overall = APPROVED

7.4 Review Session Tracking

Each fan-out cycle creates a review_sessions row (see §10). The COO tracks: -
timeout_count — incremented each time a reviewer doesn't respond within
review_timeout - status — IN_PROGRESS | COMPLETED | TIMED_OUT | CIRCUIT_OPEN

7.5 Review Circuit Breaker

If a single review session accumulates ≥ 2 **reviewer timeouts**, the session is terminated and the project transitions to FAILED:

```
# In COO's review aggregation logic:
if session.timeout_count >= 2:
    # Circuit opens – do NOT proceed with partial reviews
    controller.transition(
        project_id,
        event="review_circuit_open",
        context={"session_id": session.id, "timeouts": session.timeout_count}
    )
    # Project → FAILED(REVIEW_CIRCUIT_OPEN)
    # CEO is notified; Human can RETRY or ARCHIVE
```

Rationale: Proceeding with missing reviews creates unreviewed blind spots. Two timeouts indicate a systemic problem (agent crash, LLM outage) rather than transient slowness.

7.6 CSO Veto in Review

When a CSO REVIEW_RESPONSE includes severity: BLOCKER and veto: true:

1. COO immediately halts fan-in aggregation (does not wait for remaining reviewers)
2. COO sends the veto detail to the controller
3. Controller transitions project to SECURITY_BLOCKED sub-state
4. CEO is notified with the CSO's findings
5. Resolution: see CSO Veto Protocol in §5.6

Note: A SECURITY_BLOCKED state pauses all downstream work. No department receives new tasks until the block is resolved.

Timeout handling: If a reviewer doesn't respond within review_timeout (configurable, default 5 minutes), COO sends a reminder. After 2nd timeout within same session, circuit breaker activates (see §7.5).

8. KPI & Learning System

8.1 Metrics Collected

Metric	Computed By	When	Formula
Estimation Accuracy	CTO	End of sprint	$1 - \frac{\text{abs}(\text{estimated} - \text{actual})}{\text{estimated}}$
Task Completion Rate	CTO	End of sprint	$\frac{\text{completed_issues}}{\text{total_issues}}$
Review Pass Rate	COO	Per review cycle	$\frac{\text{approved_first_try}}{\text{total_reviews}}$
Velocity	CTO	End of sprint	$\frac{\text{story_points_completed}}{\text{sprint_duration}}$
Defect Rate	QA Lead	End of sprint	$\frac{\text{bugs_found}}{\text{features_delivered}}$
Rework Rate	COO	Per project	$\frac{\text{cdr_revision_count}}{1}$ (lower is better)
Budget Adherence	CFO	End of project	$\frac{\text{actual_cost}}{\text{estimated_cost}}$
Resource Utilization	CHRM	End of sprint	$\frac{\text{hours_worked}}{\text{hours_available}}$
Infra Lead Time	CTO	Per sprint	$\text{infra_ready_at} - \text{infra_requested_at}$

8.2 Learning Loop

Project N completes



CTO computes all KPI metrics → stores in `kpi_metrics` table



Project N+1 starts



CTO queries historical KPI:

- "For similar projects, what was avg estimation accuracy?"
- "Which agent types tend to underestimate?"
- "What velocity should I plan for?"



CTO loads `agent_profiles` for each assignee



CTO adjusts estimates: `estimate *= agent.correction_factor`



▼
Better sprint plans over time

Implementation: CTO's system prompt includes instructions to query `kpi_metrics` table and `agent_profiles` table (via tools) before creating sprint plans. The LLM uses historical data + per-agent correction factors as context for estimation. No separate ML model — the LLM's in-context learning from structured KPI data is sufficient for v1.

8.3 Agent Profile Learning

After each sprint, the CTO updates per-agent profiles in `agent_profiles` table (see §10):

```
# Per-agent rolling update (exponential moving average):
new_accuracy = actual_hours / estimated_hours # for this sprint
agent.avg_estimation_accuracy = (
    0.7 * agent.avg_estimation_accuracy + 0.3 * new_accuracy
)
agent.estimated_bias = "OVER" if new_accuracy > 1.05 else (
    "UNDER" if new_accuracy < 0.95 else "NEUTRAL"
)
agent.correction_factor = 1.0 / agent.avg_estimation_accuracy
agent.confidence = min(1.0, agent.tasks_completed / 20) # ramps up with
experience
```

This data persists across projects. The CTO reads agent profiles when assigning issues and adjusting per-agent estimates.

8.4 Sprint Retrospective (Automated)

After each sprint, CTO: 1. Fetches all SPRINT_REPORT messages from department PMs 2. Computes metrics 3. Identifies: what went well, what went wrong, specific agent underperformance 4. Adjusts next sprint plan accordingly 5. Stores retrospective as a document in MinIO (for future reference by CEO)

9. Human-in-the-Loop Integration

9.1 Approval Gate API & Controller Endpoints

The **orchestrator-api** exposes endpoints for human interaction and the deterministic workflow controller:

# Human-facing endpoints	
POST /projects	# Human creates a project request
GET /projects/{id}	# Project status + state
GET /projects/{id}/pending-decisions	# What decisions need human input
POST /projects/{id}/decisions (HumanDecision model)	# Human submits a decision
GET /projects/{id}/documents (PDR, CDR, RR)	# List all project documents

```

GET    /projects/{id}/documents/{doc_id}    # Get document details + download
link
GET    /projects/{id}/feasibility          # Get feasibility report
GET    /projects/{id}/sprints              # Sprint status and progress

# Deterministic Workflow Controller endpoints (internal – called by agents
via tools)
POST   /projects/{id}/transition            # State transition { event,
context, actor_id }
GET    /projects/{id}/allowed-transitions  # Which events are valid from
current state
GET    /projects/{id}/state-history        # Audit log of all transitions

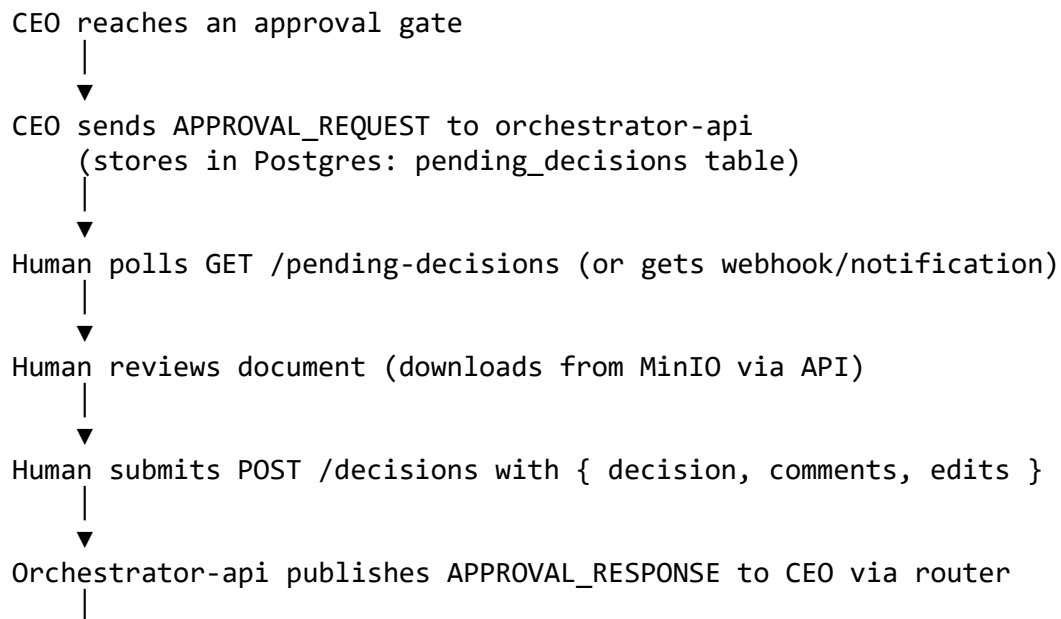
# FAILED state management (Human-facing)
POST   /projects/{id}/retry                # Reset FAILED project to last
safe state
POST   /projects/{id}/archive              # Permanently archive a FAILED
project

# Dead-letter inspection (Human-facing)
GET    /dead-letters                      # List DLQ entries (paginated)
GET    /dead-letters/{id}                 # Inspect specific dead letter
POST   /dead-letters/{id}/replay          # Re-inject into target stream

```

Controller contract: The `/transition` endpoint is the **sole writer** of `projects.state`. No agent writes directly. The controller validates `(current_state, event) → next_state` against the transition table (§11.2), persists atomically, and publishes a `SYSTEM_EVENT` notification to the relevant stream.

9.2 Decision Flow





CEO processes decision → advances project state machine

9.3 Notification (Future Enhancement)

v1: Human polls the API. v2: Add webhook URL in project config → orchestrator-api calls it when a decision is needed. Supports Slack, Discord, email integrations.

10. Enhanced Postgres Schema

Extends the infrastructure plan's schema (Phase 7a) with project/workflow tables:

```
-- Original tables from infrastructure plan (unchanged):
--   memory, task_log, artifacts

-- NEW: Projects
CREATE TABLE projects (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT NOT NULL,
  description  TEXT,
  state       TEXT NOT NULL DEFAULT 'INIT', -- State machine state
  (controller-managed)
  failure_reason TEXT, -- Set when state =
  'FAILED'
  created_by   TEXT NOT NULL, -- agent_id (CEO)
  human_requester TEXT, -- Human identifier
  config       JSONB, -- Project-level settings
  created_at   TIMESTAMPTZ DEFAULT now(),
  updated_at   TIMESTAMPTZ DEFAULT now()
);

-- NEW: Documents (PDR, CDR, RR)
CREATE TABLE documents (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id  UUID NOT NULL REFERENCES projects(id),
  doc_type    TEXT NOT NULL, -- 'PDR', 'CDR', 'RR'
  version     INT NOT NULL DEFAULT 1,
  title       TEXT NOT NULL,
  content_ref TEXT NOT NULL, -- MinIO key
  sections_meta JSONB, -- Structured summary/TOC
  status      TEXT NOT NULL DEFAULT 'DRAFT',
  created_by  TEXT NOT NULL, -- agent_id
  created_at  TIMESTAMPTZ DEFAULT now(),
  updated_at  TIMESTAMPTZ DEFAULT now(),
  UNIQUE(project_id, doc_type, version)
);

-- NEW: Reviews
```

```

CREATE TABLE reviews (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  document_id UUID NOT NULL REFERENCES documents(id),
  reviewer_id TEXT NOT NULL,           -- agent_id
  reviewer_role TEXT NOT NULL,         -- 'CFO', 'CIO', etc.
  severity    TEXT NOT NULL,           -- 'BLOCKER', 'MAJOR',
  'MINOR', 'SUGGESTION', 'APPROVED'
  section     TEXT,
  comment     TEXT NOT NULL,
  suggested_fix TEXT,
  created_at  TIMESTAMPTZ DEFAULT now()
);

```

-- NEW: Approval Gates

```

CREATE TABLE approval_gates (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id  UUID NOT NULL REFERENCES projects(id),
  gate_type   TEXT NOT NULL,           -- 'FEASIBILITY',
  'CDR_APPROVAL'
  status      TEXT NOT NULL DEFAULT 'PENDING', -- 'PENDING',
  'APPROVED', 'REJECTED', 'EDIT_REQUESTED'
  summary_ref TEXT,                   -- MinIO key for summary
  doc
  human_decision JSONB,               -- HumanDecision JSON
  requested_at TIMESTAMPTZ DEFAULT now(),
  decided_at  TIMESTAMPTZ
);

```

-- NEW: Sprints

```

CREATE TABLE sprints (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id  UUID NOT NULL REFERENCES projects(id),
  name        TEXT NOT NULL,
  milestone   TEXT,
  sprint_number INT NOT NULL,
  start_date  TIMESTAMPTZ,
  end_date    TIMESTAMPTZ,
  status      TEXT NOT NULL DEFAULT 'PLANNED',
  velocity_planned FLOAT,
  velocity_actual  FLOAT,
  created_at  TIMESTAMPTZ DEFAULT now(),
  updated_at  TIMESTAMPTZ DEFAULT now()
);

```

-- NEW: Issues

```

CREATE TABLE issues (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  sprint_id   UUID REFERENCES sprints(id),
  project_id  UUID NOT NULL REFERENCES projects(id),
  title       TEXT NOT NULL,

```

```

        description      TEXT,
        issue_type       TEXT NOT NULL,                -- 'FEATURE', 'TEST',
        'QA', 'BUGFIX', 'DOCS', 'INFRA'
        assignee_team    TEXT,                          -- team_id
        assignee_agent    TEXT,                          -- agent_id
        priority          TEXT NOT NULL DEFAULT 'MEDIUM',
        estimate_hours    FLOAT,
        actual_hours      FLOAT,
        status            TEXT NOT NULL DEFAULT 'TODO',
        parent_issue_id   UUID REFERENCES issues(id),    -- For sub-issues
        dependencies      UUID[] DEFAULT '{}',          -- Issue IDs this
depends on
        created_at        TIMESTAMPTZ DEFAULT now(),
        updated_at        TIMESTAMPTZ DEFAULT now()
    );

-- NEW: KPI Metrics
CREATE TABLE kpi_metrics (
    id                    BIGSERIAL PRIMARY KEY,
    project_id            UUID NOT NULL REFERENCES projects(id),
    sprint_id             UUID REFERENCES sprints(id),
    agent_id              TEXT NOT NULL,
    metric_type           TEXT NOT NULL,
    value                 FLOAT NOT NULL,
    context               JSONB,
    measured_at           TIMESTAMPTZ DEFAULT now()
);

-- NEW: Review Sessions (tracks fan-out/fan-in cycles)
CREATE TABLE review_sessions (
    id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    document_id           UUID NOT NULL REFERENCES documents(id),
    initiated_by          TEXT NOT NULL,                -- agent_id (COO)
    reviewer_roles        TEXT[] NOT NULL,              --
    {'CFO', 'CIO', 'CHRM', 'CSO'}
    status                TEXT NOT NULL DEFAULT 'IN_PROGRESS', -- IN_PROGRESS |
COMPLETED | TIMED_OUT | CIRCUIT_OPEN
    timeout_count         INT NOT NULL DEFAULT 0,
    due_at                TIMESTAMPTZ NOT NULL,          -- review_timeout
deadline
    completed_at          TIMESTAMPTZ,
    created_at            TIMESTAMPTZ DEFAULT now()
);

-- NEW: Agent Profiles (per-agent rolling performance data)
CREATE TABLE agent_profiles (
    agent_id              TEXT PRIMARY KEY,
    team_id               TEXT NOT NULL,
    role                  TEXT NOT NULL,
    avg_estimation_accuracy FLOAT NOT NULL DEFAULT 1.0,

```

```

    estimation_bias          TEXT NOT NULL DEFAULT 'NEUTRAL', -- OVER | UNDER
    / NEUTRAL
    correction_factor        FLOAT NOT NULL DEFAULT 1.0,
    confidence               FLOAT NOT NULL DEFAULT 0.0,      -- 0.0 to 1.0
    tasks_completed          INT NOT NULL DEFAULT 0,
    last_active_project      UUID REFERENCES projects(id),
    updated_at               TIMESTAMPTZ DEFAULT now()
);

-- NEW: Dead Letters (messages that exhausted retries or expired TTL)
CREATE TABLE dead_letters (
    id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    original_msg_id        UUID NOT NULL,                    --
    MessageEnvelope.message_id
    stream_name            TEXT NOT NULL,                    -- e.g.,
    'stream:exec_ceo'
    consumer_group         TEXT NOT NULL,
    envelope               JSONB NOT NULL,                  -- Full MessageEnvelope
    JSON
    failure_reason         TEXT NOT NULL,                    -- 'MAX_RETRIES' |
    'TTL_EXPIRED'
    retry_count            INT NOT NULL DEFAULT 0,
    created_at             TIMESTAMPTZ DEFAULT now()
);

-- NEW: Project State History (audit log for controller transitions)
CREATE TABLE project_state_history (
    id                     BIGSERIAL PRIMARY KEY,
    project_id            UUID NOT NULL REFERENCES projects(id),
    from_state            TEXT NOT NULL,
    to_state              TEXT NOT NULL,
    event                 TEXT NOT NULL,                    -- e.g., 'pdr_submitted',
    'human_approved'
    actor_id              TEXT NOT NULL,                    -- agent_id that
    triggered
    context                JSONB,                            -- Optional metadata
    transitioned_at       TIMESTAMPTZ DEFAULT now()
);

-- NEW: Infrastructure Events (DevOps INFRA_READY tracking)
CREATE TABLE infra_events (
    id                     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id            UUID NOT NULL REFERENCES projects(id),
    sprint_id             UUID REFERENCES sprints(id),
    event_type            TEXT NOT NULL,                    -- 'INFRA_REQUESTED' |
    'INFRA_READY' | 'INFRA_FAILED'
    details               JSONB,
    created_at            TIMESTAMPTZ DEFAULT now()
);

```

-- Indexes

```
CREATE INDEX idx_documents_project ON documents(project_id);
CREATE INDEX idx_reviews_document ON reviews(document_id);
CREATE INDEX idx_approval_gates_project ON approval_gates(project_id);
CREATE INDEX idx_sprints_project ON sprints(project_id);
CREATE INDEX idx_issues_sprint ON issues(sprint_id);
CREATE INDEX idx_issues_project ON issues(project_id);
CREATE INDEX idx_issues_assignee ON issues(assignee_team, assignee_agent);
CREATE INDEX idx_kpi_project ON kpi_metrics(project_id);
CREATE INDEX idx_kpi_agent ON kpi_metrics(agent_id, metric_type);
CREATE INDEX idx_kpi_type_date ON kpi_metrics(metric_type, measured_at);
CREATE INDEX idx_review_sessions_doc ON review_sessions(document_id);
CREATE INDEX idx_dead_letters_msg ON dead_letters(original_msg_id);
CREATE INDEX idx_dead_letters_created ON dead_letters(created_at);
CREATE INDEX idx_state_history_project ON project_state_history(project_id);
CREATE INDEX idx_state_history_time ON
project_state_history(transitioned_at);
CREATE INDEX idx_infra_events_project ON infra_events(project_id);
```

PgBouncer note: All connections go through PgBouncer (transaction mode).
asyncpg must use `statement_cache_size=0` in the DSN to avoid prepared-
statement conflicts. See infrastructure plan §Phase 7a.

11. Integration with MAS Infrastructure Plan

This section maps every element of the organizational architecture onto the existing infrastructure plan phases.

11.1 Phase Modifications

Infrastructure Phase	What Changes
Phase 0 — Scaffold	Add teams/ YAML for all 11 teams (exec_ceo, exec_coo, office_cfo, office_cio, office_chrm, office_cso, office_cto, dept_production, dept_system, dept_qa, dept_devops). Add packages/mas-core/workflow/ for deterministic controller.
Phase 1 — Protocols	Use unified MessageEnvelope (§3.1). Add all expanded MessageType values (§3.2). Add AgentProfile, BlobRef models.
Phase 2 — Policy	Rewrite CommunicationPolicy with 6 roles instead of 4. Add full hierarchy matrix (§4). Add tool-

	permission enforcement.
Phase 3 — Router	Add Redis Streams hardening (§4.4): XAUTOCLAIM, DLQ, publish/consume idempotency, stream trimming, Redis ACL.
Phase 4 — Agent Runtime	Add ExecutiveAgent (COO), CSuiteAgent (CFO/CIO/CHRM/CSO/CTO). CEO uses orchestrator runtime with controller integration.
Phase 5 — LLM Gateway	No changes. All agent types use the same LLM gateway.
Phase 6 — Tool Service	Add tool manifest with 6 groups (§D below). Role-based permission matrix. Circuit breaker per tool.
Phase 7 — Storage	Add 11 new Postgres tables (§10). Add Alembic migrations. PgBouncer statement_cache_size=0. MinIO: add /retrospectives/ bucket path.
Phase 8 — Agent Types	Add ExecutiveAgent, CSuiteAgent alongside existing WorkerAgent, AdminAgent, SubAgent.
Phase 9 — Team Runner	Each team YAML specifies corporate role. Team runner passes role to agent constructor. 11 team YAMLs.
Phase 10 — Orchestrator API	Add controller endpoints (§9.1). DLQ management. FAILED state retry/archive.
Phase 11 — Compose	11 team-runner containers + 7 infra = 18 containers . Apply mem_limit per team.
Phase 12 — Observability	Add: mas_project_state, mas_dlq_depth, mas_review_circuit_open, mas_infra_lead_time, mas_agent_correction_factor.

11.2 New Phase: Phase 4b — Deterministic Workflow Controller (packages/mas-core/workflow/)

Insert between Phase 4 (Agent Runtime) and Phase 5 (LLM Gateway):

WorkflowController: A deterministic state machine inside orchestrator-api that is the **sole writer** of `projects.state`. Agents emit events; the controller validates and persists transitions atomically.

```
class WorkflowController:
    """Deterministic state-transition engine.
    Lives inside orchestrator-api. Not an LLM agent – pure Python Logic."""

    def __init__(self, db: AsyncSession):
        self.db = db

    async def transition(
        self, project_id: UUID, event: str, actor_id: str, context: dict |
None = None
    ) -> str:
        """
        1. Read current state from Postgres (SELECT ... FOR UPDATE)
        2. Look up (current_state, event) in TRANSITIONS
        3. If not found → raise InvalidTransition (400)
        4. Write new state + append to project_state_history (single
transaction)
        5. Publish SYSTEM_EVENT to stream:{relevant_team}
        6. Return new state
        """
        ...

    TRANSITIONS: dict[tuple[str, str], str] = {
        # Happy path (14-step workflow)
        ("INIT", "project_created"): "FEASIBILITY_CHECK",
        ("FEASIBILITY_CHECK", "all_reviews_in"): "FEASIBILITY_REPORT",
        ("FEASIBILITY_REPORT", "human_approved"): "PDR_CREATION",
        ("FEASIBILITY_REPORT", "human_rejected"): "ARCHIVED",
        ("PDR_CREATION", "pdr_submitted"): "PDR_REVIEW",
        ("PDR_REVIEW", "all_reviews_in"): "CDR_CREATION",
        ("CDR_CREATION", "cdr_submitted"): "CDR_REVIEW",
        ("CDR_REVIEW", "cdr_presented"): "HUMAN_APPROVAL",
        ("HUMAN_APPROVAL", "human_approved"): "RR_CREATION",
        ("HUMAN_APPROVAL", "human_edits"): "CDR_CREATION", #
revision loop
        ("HUMAN_APPROVAL", "human_cancelled"): "ARCHIVED",
        ("RR_CREATION", "rr_submitted"): "SPRINT_PLANNING",
        ("SPRINT_PLANNING", "sprints_created"): "INFRA_PROVISIONING",
        ("INFRA_PROVISIONING", "infra_ready"): "IN_PROGRESS",
        ("IN_PROGRESS", "all_sprints_done"): "RETROSPECTIVE",
        ("RETROSPECTIVE", "retrospective_done"): "KPI_PERSISTENCE",
```

```

("KPI_PERSISTENCE", "kpi_saved"):          "COMPLETED",
("COMPLETED", "archive_requested"):        "ARCHIVED",

# Security veto path
("FEASIBILITY_CHECK", "cso_veto"):          "SECURITY_BLOCKED",
("PDR_REVIEW", "cso_veto"):                 "SECURITY_BLOCKED",
("CDR_REVIEW", "cso_veto"):                 "SECURITY_BLOCKED",
("SECURITY_BLOCKED", "blocker_resolved"):    "_RESTORE_PRIOR_STATE",
("SECURITY_BLOCKED", "ceo_override"):        "_RESTORE_PRIOR_STATE",

# FAILED paths
("...", "watchdog_timeout"):                 "FAILED", # any state →
FAILED
("...", "review_circuit_open"):              "FAILED",
("...", "unrecoverable_error"):              "FAILED",
("INFRA_PROVISIONING", "infra_failed"):      "FAILED",

# Recovery from FAILED
("FAILED", "retry"):
"_RESTORE_LAST_SAFE_STATE",
("FAILED", "archive_requested"):             "ARCHIVED",
}

# Watchdog: cron job runs every 60s, checks for projects stuck in
# same state > 1 hour → fires ("...", "watchdog_timeout")

```

Key difference from previous ProjectStateMachine: The controller lives in orchestrator-api (not in any agent's process). Agents call POST /projects/{id}/transition with an event. The controller is **not** an LLM — it is pure deterministic Python. This makes state transitions restart-proof and auditable via project_state_history.

11.2.1 Tool Service Manifest

The tool-service exposes tools in **6 groups**. Each tool is gated by (sender_role, tool_name) validation.

Group	Tools	Accessible By
Workflow	project.create, project.status, project.transition , project.list	orchestrator, executive, c_suite (transition: orchestrator only)
Document	document.create_draft, document.submit, document.revise, document.get_latest, document.list	executive, c_suite, admin, worker (create/submit: admin+)

Review	review.submit, review.aggregate, review.submit_veto , review.start_session	c_suite (submit), executive (aggregate, start_session)
Sprint / Issue	sprint.create, sprint.activate, issue.create, issue.decompose, issue.update_status, issue.list	c_suite:cto (create/activate), admin (update_status), worker (update_status own)
DevOps	infra.provision, cicd.configure, monitoring.setup, secrets.manage, infra.ready_signal	admin:devops_pm, worker:devops
KPI / Utility	kpi.compute, kpi.query_history, kpi.update_agent_profile, velocity.report, estimation.adjust, blob.*, web_search, web_fetch	c_suite:cto (kpi.*), all roles (blob.download, web_search)

Tool reliability: - **Token bucket:** Each tool group has a rate limit (e.g., sprint.* → 20 calls/min) to prevent runaway loops - **Cache / dedupe:** Identical tool calls within 30 s return cached result (keyed by (tool_name, args_hash)) - **Circuit breaker per tool:** If a tool fails ≥ 3 times in 60 s, it is marked OPEN for 120 s (returns error immediately). After cooldown, one probe call is allowed (HALF_OPEN). On success → CLOSED.

11.3 Team YAML Examples

```
# teams/exec_ceo.yml
team_id: exec_ceo
team_name: "CEO Office"
admin:
  agent_id: ceo
  role: orchestrator
  model: gpt-4o # Or your preferred model via LLM gateway
  system_prompt_template: prompts/ceo.md
  budget_defaults:
    max_llm_calls: 200
    max_tool_calls: 50
    max_cost_usd: 5.0
  tools:
    - project_create
```

```
    - project_status
    - human_notify
    - human_await_decision
    - feasibility_aggregate
workers: []
```

```
---
# teams/exec_coo.yml
team_id: exec_coo
team_name: "COO Office"
admin:
  agent_id: coo
  role: executive
  model: gpt-4o
  system_prompt_template: prompts/coo.md
  budget_defaults:
    max_llm_calls: 300
    max_tool_calls: 100
    max_cost_usd: 8.0
  tools:
    - document_create
    - document_get
    - review_aggregate
    - department_task
    - project_status_update
workers: []
```

```
---
# teams/office_cfo.yml
team_id: office_cfo
team_name: "CFO Office"
admin:
  agent_id: cfo
  role: c_suite
  model: gpt-4o
  system_prompt_template: prompts/cfo.md
  budget_defaults:
    max_llm_calls: 100
    max_tool_calls: 30
    max_cost_usd: 2.0
  tools:
    - cost_estimate
    - budget_check
    - roi_calculate
    - web_search
workers:
  - agent_id: financial_analyst_1
    role: worker
    model: gpt-4o-mini      # Cheaper model for analysis sub-tasks
    system_prompt_template: prompts/financial_analyst.md
```

```

    budget_defaults:
      max_llm_calls: 50
      max_tool_calls: 20
      max_cost_usd: 1.0

---
# teams/dept_production.yml
team_id: dept_production
team_name: "Production Department"
admin:
  agent_id: production_pm
  role: admin
  model: gpt-4o
  system_prompt_template: prompts/production_pm.md
  budget_defaults:
    max_llm_calls: 150
    max_tool_calls: 50
    max_cost_usd: 4.0
  tools:
    - document_create
    - document_get
    - blob_upload
workers:
  - agent_id: requirements_writer
    role: worker
    model: gpt-4o
    system_prompt_template: prompts/requirements_writer.md
    budget_defaults:
      max_llm_calls: 100
      max_tool_calls: 30
      max_cost_usd: 3.0
  - agent_id: planner
    role: worker
    model: gpt-4o-mini
    system_prompt_template: prompts/planner.md
    budget_defaults:
      max_llm_calls: 80
      max_tool_calls: 20
      max_cost_usd: 1.5
  - agent_id: cost_estimator
    role: worker
    model: gpt-4o-mini
    system_prompt_template: prompts/cost_estimator.md
    budget_defaults:
      max_llm_calls: 60
      max_tool_calls: 20
      max_cost_usd: 1.0

---
# teams/office_cto.yml

```

```

team_id: office_cto
team_name: "CTO Office"
admin:
  agent_id: cto
  role: c_suite
  model: gpt-4o
  system_prompt_template: prompts/cto.md
  budget_defaults:
    max_llm_calls: 200
    max_tool_calls: 100
    max_cost_usd: 5.0
  tools:
    - sprint_create
    - issue_create
    - issue_decompose
    - kpi_compute
    - kpi_query
    - velocity_report
    - estimation_adjust
workers:
  - agent_id: sprint_planner
    role: worker
    model: gpt-4o-mini
    system_prompt_template: prompts/sprint_planner.md
    budget_defaults:
      max_llm_calls: 80
      max_tool_calls: 50
      max_cost_usd: 2.0
  - agent_id: kpi_analyst
    role: worker
    model: gpt-4o-mini
    system_prompt_template: prompts/kpi_analyst.md
    budget_defaults:
      max_llm_calls: 60
      max_tool_calls: 40
      max_cost_usd: 1.5

```

DevOps Team YAML

```

# teams/dept_devops.yml
team_id: dept_devops
team_name: "DevOps Department"
admin:
  agent_id: devops_pm
  role: admin
  model: gpt-4o
  system_prompt_template: prompts/devops_pm.md
  budget_defaults:
    max_llm_calls: 150
    max_tool_calls: 80
    max_cost_usd: 4.0

```

```

tools:
  - infra.provision
  - cicd.configure
  - monitoring.setup
  - secrets.manage
  - infra.ready_signal
  - blob.upload
  - blob.download
workers:
  - agent_id: devops_eng
    role: worker
    model: gpt-4o-mini
    system_prompt_template: prompts/devops_eng.md
    budget_defaults:
      max_llm_calls: 100
      max_tool_calls: 60
      max_cost_usd: 2.0
  - agent_id: sre_agent
    role: worker
    model: gpt-4o-mini
    system_prompt_template: prompts/sre_agent.md
    budget_defaults:
      max_llm_calls: 80
      max_tool_calls: 50
      max_cost_usd: 1.5

```

11.4 Updated Docker Compose Services

```

services:
  # Infrastructure (unchanged from infra plan)
  redis: ...
  postgres: ...
  pgbouncer: ...
  minio: ...
  message-router: ...
  tool-service: ...
  orchestrator-api: ...

  # C-Suite teams
  team_ceo:
    build: { context: ., dockerfile: infra/docker/Dockerfile.team-runner }
    environment:
      TEAM_CONFIG: /teams/exec_ceo.yml
    mem_limit: 512m
    cpus: 0.5

  team_coo:
    build: { context: ., dockerfile: infra/docker/Dockerfile.team-runner }
    environment:
      TEAM_CONFIG: /teams/exec_coo.yml
    mem_limit: 512m

```



```
    cpus: 0.5

team_cfo:
  build: { context: ., dockerfile: infra/docker/Dockerfile.team-runner }
  environment:
    TEAM_CONFIG: /teams/office_cfo.yml
  mem_limit: 384m
  cpus: 0.3

team_cio:
  build: { context: ., dockerfile: infra/docker/Dockerfile.team-runner }
  environment:
    TEAM_CONFIG: /teams/office_cio.yml
  mem_limit: 384m
  cpus: 0.3

team_chrm:
  build: { context: ., dockerfile: infra/docker/Dockerfile.team-runner }
  environment:
    TEAM_CONFIG: /teams/office_chrm.yml
  mem_limit: 384m
  cpus: 0.3

team_cso:
  build: { context: ., dockerfile: infra/docker/Dockerfile.team-runner }
  environment:
    TEAM_CONFIG: /teams/office_cso.yml
  mem_limit: 384m
  cpus: 0.3

team_cto:
  build: { context: ., dockerfile: infra/docker/Dockerfile.team-runner }
  environment:
    TEAM_CONFIG: /teams/office_cto.yml
  mem_limit: 512m
  cpus: 0.5

# Department teams
team_production:
  build: { context: ., dockerfile: infra/docker/Dockerfile.team-runner }
  environment:
    TEAM_CONFIG: /teams/dept_production.yml
  mem_limit: 768m
  cpus: 0.8

team_system:
  build: { context: ., dockerfile: infra/docker/Dockerfile.team-runner }
  environment:
    TEAM_CONFIG: /teams/dept_system.yml
```

```

    mem_limit: 768m
    cpus: 0.8

team_qa:
  build: { context: ., dockerfile: infra/docker/Dockerfile.team-runner }
  environment:
    TEAM_CONFIG: /teams/dept_qa.yml
  mem_limit: 512m
  cpus: 0.5

team_devops:
  build: { context: ., dockerfile: infra/docker/Dockerfile.team-runner }
  environment:
    TEAM_CONFIG: /teams/dept_devops.yml
  mem_limit: 512m
  cpus: 0.5

```

Total: 11 team containers + 7 infra containers = 18 containers

Resource estimate (all teams idle between tasks; only active containers consume LLM tokens): - Memory: ~6 GB for teams + ~2 GB for infra ≈ **8 GB minimum** - CPUs: ~6 cores for teams + ~2 cores for infra ≈ **8 cores recommended**

12. Updated Implementation Order

Phase 0 + 1	Scaffold + protocols (unified MessageEnvelope, all models)
Phase 2	Policy engine (6-role hierarchy + tool permissions)
Phase 4b watchdog)	Deterministic Workflow Controller (transition table,
Phase 5	LLM gateway
Phase 3 ACL)	Message router (Redis Streams hardening, XAUTOCLAIM, DLQ,
Phase 6	Tool service (6 groups, role-gated, circuit breakers)
Phase 7	Storage (original tables + 11 new tables, PgBouncer fix)
Phase 4 + 8	Agent runtime + 5 agent types (WorkerAgent, AdminAgent, SubAgent, ExecutiveAgent [new], CSuiteAgent [new])
Phase 9	Team runner (11 team YAMLS incl. DevOps)
Phase 10 the-loop)	Orchestrator API (controller endpoints, DLQ mgmt, Human-in-

Phase 11	Docker Compose (18 containers)
Phase 12	Observability + KPI dashboards + DLQ alerts

13. Enhanced Verification

Add to the existing test suite:

Workflow & Controller tests: - **Happy-path workflow:** Submit project → verify it traverses all 14 states (INIT → ... → COMPLETED → ARCHIVED). Assert every transition is logged in `project_state_history`. - **Invalid transition:** Call POST `/projects/{id}/transition` with an event not valid for the current state → assert 400 error, state unchanged. - **Watchdog timeout:** Set project in `PDR_CREATION`, wait >1 hour (or mock time) → assert watchdog fires and project → `FAILED`. - **FAILED retry:** Set project to `FAILED` → call POST `/projects/{id}/retry` → assert project restored to last safe state. - **FAILED archive:** Set project to `FAILED` → call POST `/projects/{id}/archive` → assert project → `ARCHIVED`.

Review & Security tests: - **Feasibility fan-out:** CEO sends `REVIEW_REQUEST` to 4 C-Suite → all respond → verify aggregation and state advance. - **Review circuit breaker:** Start review session → 2 reviewers timeout → assert `review_sessions.status = CIRCUIT_OPEN` and project → `FAILED(REVIEW_CIRCUIT_OPEN)`. - **CSO veto:** CSO submits `REVIEW_RESPONSE` with `severity: BLOCKER, veto: true` → assert project → `SECURITY_BLOCKED`. Then: (a) test CEO override, (b) test blocker resolution. - **CDR revision loop:** Human requests edits → assert state loops back to `CDR_CREATION` → System dept re-tasked.

Message & Router tests: - **Publish idempotency:** Send same `message_id` twice → assert second is silently discarded (dedupe key in Redis). - **Consume idempotency:** Redeliver a message via `XAUTOCLAIM` → assert consumer's LRU cache prevents double-processing. - **XAUTOCLAIM reclaim:** Leave a message unacked for >120 s → assert `XAUTOCLAIM` picks it up and increments `retry_count`. - **DLQ entry:** Send message that fails 3 times → assert row in `dead_letters` table, stream entry removed. - **Chain of command:** Worker messages CEO directly → router rejects with policy violation. - **Escalation path:** Worker → PM → COO → CEO escalation chain verified.

DevOps & Infrastructure tests: - **INFRA_READY gate:** After `SPRINT_PLANNING`, assert dev sprint issues cannot start until DevOps PM sends `INFRA_READY`. - **INFRA timeout:** DevOps exceeds SLA (mock 30 min) → assert escalation to CTO → CEO. - **Sprint decomposition:** Submit RR → CTO creates sprints with INFRA-type issues assigned to `dept_devops` → verify.

KPI & Learning tests: - **KPI persistence:** Complete project → assert `kpi_metrics` rows for all metric types. - **Agent profile update:** Complete 2 sprints → assert

agent_profiles.correction_factor updated. - **KPI learning**: On project N+1, CTO queries agent profiles → verify correction factors influence estimates.

Tool service tests: - **Role-gated access**: Worker calls project.transition → tool-service rejects (403). - **Circuit breaker**: Fail tool 3x in 60 s → assert next call returns OPEN error immediately. After 120 s → HALF_OPEN. - **Token bucket**: Exceed rate limit for sprint.* group → assert 429 response.

Integration tests: - **18-container compose**: docker compose up on 8 GB / 4 core machine → all containers healthy within 90 s. - **Cancel test**: Human cancels at any approval gate → project → ARCHIVED, all pending tasks cancelled. - **Review timeout with circuit breaker**: One reviewer doesn't respond → COO sends reminder → 2nd timeout → circuit breaker activates.

14. Decisions Summary

Decision	Choice	Rationale
Agent hierarchy	Corporate C-Suite + Departments	Mirrors real org structure; clear accountability
CEO role	orchestrator (MAS role)	Only agent that talks to Human; global access
COO role	executive (new MAS role)	Needs cross-dept + cross-C-Suite access; distinct from orchestrator
C-Suite role	c_suite (new MAS role)	Peer-to-peer review access; restricted cross-team
Workflow controller	Deterministic engine in orchestrator-api	Agents emit events, controller validates + persists atomically; restart-proof
FAILED state	Explicit with failure_reason enum	Watchdog, circuit breaker, infra failure all funnel to one recoverable state
CSO veto	SECURITY_BLOCKED sub-state	Security has hard-stop power; only CEO override can resume (audited)
DevOps department	v1 critical path (not future)	INFRA_READY gate required before dev sprints; CI/CD is essential
Message protocol	Unified MessageEnvelope (replaces dual schemas)	Single canonical format; 64 KB limit + BlobRef for large payloads
Redis Streams	XAUTOCLAIM + DLQ +	Restart-proof delivery; no lost

	ACL + idempotency	messages; least-privilege security
Tool service	6 groups, role-gated, circuit breakers	Prevents runaway loops; clear permission boundary per role
Agent profiles	Per-agent <code>correction_factor</code> in Postgres	Estimation learning survives restarts; no separate ML model
Document lifecycle	PDR → CDR → RR with versioning	Structured quality gates before execution
Human-in-the-loop	API polling (v1), webhook (v2)	Simple first, add push notifications later
KPI learning	LLM in-context from Postgres data	No separate ML; LLM reads historical KPIs + agent profiles
Review fan-out	Parallel via router, circuit breaker on ≥2 timeouts	Don't block forever; fail fast on systemic issues
Sprint management	CTO as dedicated agent with workers	Separation of planning from execution
Team count	11 teams (7 C-Suite offices + 4 departments)	Each role gets isolated container; scales independently
Agent models	gpt-4o for leads, gpt-4o-mini for workers	Cost optimization: cheap models for routine tasks

15. Future Enhancements (v2+)

1. **Dynamic department scaling:** CTO/CHRM analyze workload → spin up additional worker containers via Docker API
2. **Multi-project support:** Multiple projects running concurrently with resource contention management by CHRM
3. **Inter-project knowledge transfer:** KPI insights from project A inform project B estimates
4. **Dashboard UI:** Real-time project status, Gantt charts, KPI graphs (Grafana + custom frontend)
5. **Voice/chat interface:** Replace API polling with Slack/Discord bot for human interaction
6. **Auto-tuning:** CTO automatically adjusts agent `budget_defaults` based on historical performance data
7. **Approval delegation:** CEO can delegate approval authority to COO for low-risk decisions
8. **Parallel project tracks:** CDR work can start on approved sections while other sections are revised

9. **Agent performance reviews:** CHRM periodically evaluates agent KPIs and recommends model upgrades/downgrades
10. **Webhook notifications:** orchestrator-api calls webhook URL when human decision needed (Slack, Discord, email)
11. **Advanced DLQ analytics:** Dashboard for dead-letter patterns, auto-remediation suggestions